Still Needed, Hard to Transfer: Using and Reusing Contextual Skills

Anonymous Author(s)

Abstract

Language models can adapt to new tasks by following instructions provided in context. Such instructions can be organized as reusable skills, raising the possibility of sharing the internal states they produce across executions. It remains unclear how the transferability of these states relates to continued dependence on a skill across model layers. We compare two interventions across layers: transferring hidden states at skill positions from a correct-skill run into an unrelated-skill run, and blocking attention to the skill in the original run. We also test whether copying skill-position key/value caches, which store information used by attention, can improve transfer. Across four models and two benchmarks, transfer weakens before dependence on the skill does in all eight settings. At layer 16, adding caches improves accuracy over state transfer alone by 6.8 to 43.4 percentage points. Blocking skill reading in subsequent layers reduces the recovered accuracy in each setting, while copying only skill-position caches, even across all layers, falls short of correct-skill accuracy. These findings distinguish continued skill use from state reuse and show that cached attention information supports partial recovery.

CCS Concepts

• **Computing methodologies** → **Natural language processing**. *Knowledge representation and reasoning*.

Keywords

agent skills, Web agents, large language models, in-context learning, activation patching, context reuse, mechanistic interpretability

ACM Reference Format:

Anonymous Author(s). 2027. Still Needed, Hard to Transfer: Using and Reusing Contextual Skills. In *Proceedings of the ACM Web Conference 2027 (WWW '27)*. ACM, New York, NY, USA, 12 pages.

Relevance Statement. This work is relevant to the Web Infrastructure and Agentic Systems track. Web agents use reusable task instructions, or skills, across requests and services. We examine when skill-state transfer weakens despite the original run's continued skill dependence, and how attention caches partly restore transfer. These findings inform the evaluation of skill reuse and context management for Web agents.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

WWW '27, Dublin, Ireland

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

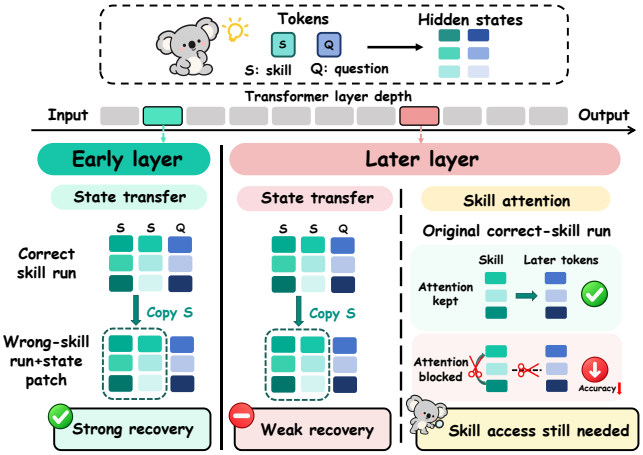


Figure 1: Weak transfer and continued skill use at the same later depth. Copying skill-position states from a correct-skill run into an unrelated-skill run gives strong early recovery (left) and weak late recovery (middle). At that same later depth, the original run still needs skill access: retaining access preserves performance, while cutting it reduces accuracy (right). S and Q label skill and question tokens. Figure 3 gives the measured depth profiles.

1 Introduction

Language models can perform new tasks by following instructions provided in context. Reusable skills organize such instructions into descriptions of how to carry out a task [32]. Web agents use skills and workflows to guide actions across requests and services [28, 29]. A calculator skill, for example, specifies a formula, input conventions and steps for applying them to a new question. As a model processes a skill, it forms hidden states, internal vectors associated with the instruction tokens. Reusing these states could allow one execution to supply task-relevant information to another without changing the model's weights.

Prior work transfers task information through internal vectors derived from demonstrations or attention outputs [13, 24]. Layer-wise interventions separately examine where state replacement loses its effect and where attention to context becomes unnecessary [20, 22]. Restoring performance by copying states and disrupting it by blocking attention, however, measure different effects [12]. A failed transfer does not establish that the original execution no longer needs the skill. We ask: *Can skill-state transfer weaken with depth while the original execution remains dependent on the skill?*

We study this question through controlled question answering with skill selection held fixed. For the same question, we compare a correct-skill run with a run using length-matched text from an unrelated skill. At each layer, the transfer test copies the correct-skill run's hidden states at skill positions into the unrelated-skill

run. Separately, the dependence test blocks attention to the skill in the correct-skill run from that layer onward. The tests use common questions within each model and task setting. Content, position and cross-question controls assess whether early transfer carries skill-relevant information.

Hidden-state copying does not directly replace the key/value (KV) caches that attention uses to retain information from processed tokens. We extend transfer by copying skill-position caches alongside hidden states and test whether the recovered execution still needs skill reading in later layers. Cache-only controls replace skill-position caches at every layer.

Across four models and two benchmarks, all eight comparisons show that transfer weakens at later layers while the original run remains strongly dependent on the skill (Figure 1).

At layer 16, joint state and cache replacement improves accuracy over states alone by 6.8 to 43.4 percentage points, and blocking subsequent skill reading reduces the recovered accuracy in every setting. Copying only skill-position caches at every layer still falls short of correct-skill accuracy. Copying caches for all processed prompt tokens reproduces the correct-skill run's generated sequence. Recomputing the later prompt with correct-skill caches also brings accuracy close to that of the correct-skill run.

Our contributions are summarized as follows:

- **Skill dependence persists as state transfer weakens.** Across all eight settings, transfer weakens at later layers while the original run remains dependent on the skill.
- **Skill caches improve state transfer.** Adding skill-position KV at layer 16 improves accuracy in all eight settings. The recovered run still depends on skill reading.
- **Skill caches alone do not restore correct-skill accuracy.** With all skill-position caches restored, copying the remaining prefix caches or recomputing the later prompt improves recovery.

2 Related Work

State transfer and its interpretation. Task vectors summarize demonstrations as internal vectors [13], and function vectors identify attention outputs that transmit task behaviour [24]. Instructions also yield transferable components [2, 6], whose effects vary with intervention depth [18]. Activation patching, which copies internal states between executions, requires matched controls and careful implementation [12, 33]. In particular, restoring behaviour and disrupting it need not reveal the same components. Interactions with other components can also affect a patching estimate [26]. We use these methodological distinctions to compare skill-state transfer and source dependence across depth.

Depth, source access and context caches. Pola and Balasubramanian [20] locate an onset beyond which selected hidden-state replacements largely cease to change answers in their instruction tasks. Sia et al. [22] use layer-wise context masking, which blocks attention to selected context positions, in machine translation and code generation to locate where attention to the instructional context becomes unnecessary. Joint transfer and masking analyses also locate subtask computation [31]. We instead define the source as an explicit skill-instruction span and compare transfer into a wrong-skill execution with blocking attention to that span in the original execution on common questions.

PrefillNotes [16] studies editable and reusable KV caches. In its field-editing tasks, replacing local KV entries can fail when downstream cached conclusions still reflect the old field. It also composes precompiled skill caches into new contexts. We relate state transfer to continued skill dependence across layers, then test joint state and source-cache transfer.

Skills and reusable context. Agent Skills organize instructions and resources for on-demand use [32]. Benchmarks study skill retrieval and application [11, 17, 23], while matched-budget evaluations expose their dependence on available computation [10]. We hold skill selection fixed. Other approaches compress context into aggregated activations [1], learned summary tokens [19], hidden states [8], or weight updates [3, 4]. Our interventions characterize naturally occurring states and source dependence without training a compression mechanism.

3 Problem Formulation

Skills, executions and token positions. We supply instructions directly, holding skill selection fixed. For the same question, the *donor* uses the correct skill and the *receiver* uses length-matched text from a fixed unrelated skill (Appendix A.1). Skill positions form the *source span* $\mathcal{I}$, including instructions such as a calculator's formula but excluding the question. *Readers* $\mathcal{J}$ are subsequent prompt and generated-token positions. Receiver denotes an execution, while reader denotes a token position. Causal attention blocks access to future tokens, so earlier positions cannot read the skill.

Layers are indexed from zero. The model processes prompt tokens during *prefill* and generates answer tokens during *decoding*. We use greedy decoding, which selects the highest-probability next token. We compare what source-state transfer restores in the receiver with what blocking source attention removes from the donor.

Hidden-state transfer. Each Transformer block updates the token vectors. Let H^L be its output at layer L, also called residual states, with subscripts D/R for donor/receiver. State replacement S writes

$$S: \qquad H_R^L[\mathcal{I}] \leftarrow H_D^L[\mathcal{I}]. \tag{1}$$

During prefill, the receiver continues from this write, retaining other positions' states at layer L.

Blocking attention to the skill. Cumulative source-attention knockout blocks access to source positions from layer L onward in the original correct-skill run. Attention uses query vectors to score keys and combines value vectors with the resulting weights. For the query-to-key score z_{ij}^{ℓ}, including the causal mask, each layer $\ell \geq L$ uses

$$\widetilde{z}_{ij}^{\ell} = \begin{cases} -\infty, & j \in \mathcal{I}, \\ z_{ij}^{\ell}, & j \notin \mathcal{I}. \end{cases} \tag{2}$$

Softmax converts these scores into weights, assigning zero to blocked positions and normalizing the rest. The cut applies to all query positions and all heads, the block's parallel attention components, during prefill and decoding. It blocks both skill reading and attention within the skill, measuring aggregate source-attention dependence.

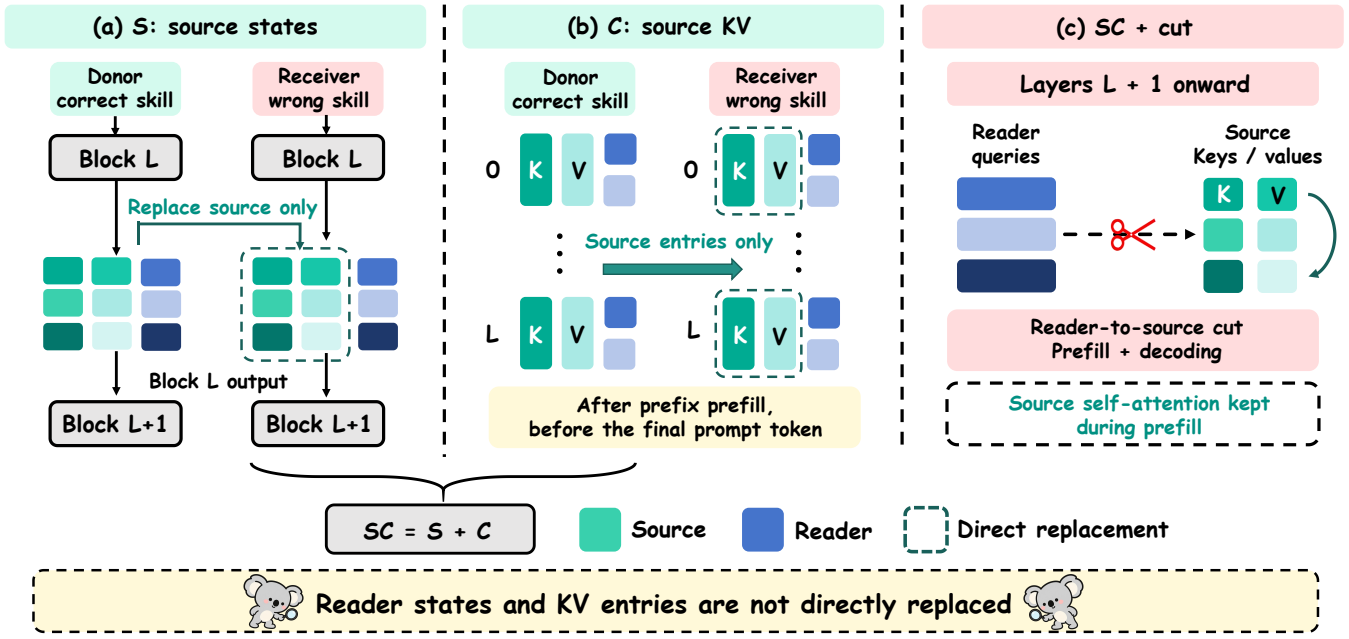


Figure 2: Receiver interventions and their timing. (a) S replaces source states at block L during prefix prefill. (b) C copies source KV at layers 0 through L after prefix prefill. SC combines both writes before recomputing the final prompt token and generating. (c) SC + cut installs its mask before prefill and blocks skill reading from $L+1$ during prefill and decoding, retaining within-skill attention. Reader states and KV are not directly replaced. Figure 4(b) instead cuts attention in the original correct-skill run.

Figure 2 also introduces source-cache replacement (C) and joint repair (SC), defined below.

Recovery and dependence. Write a_D, a_R and a_I for answer accuracy under the donor, receiver and an intervention I. Recovery is the fraction of the donor's gain restored in the receiver:

$$\rho(I) = \frac{a_I - a_R}{a_D - a_R}. \tag{3}$$

For $a_D > a_R$, $\rho = 1$ restores the gain and $\rho > 1$ exceeds donor accuracy. We write $\rho(L)$ for state replacement at layer L. All accuracies use the same items and the transfer protocol's own baselines (Section 4.1).

For knockout, let $q(L)$ be the fraction of eligible, originally correct answers that remain correct. Source-attention dependence is

$$d(L) = 1 - q(L). \tag{4}$$

Higher d means more correct answers lost. We compare receiver recovery ρ and donor dependence d on common eligible IDs, retaining each test's baselines. A paired-outcome check asks whether transfer and knockout both yield incorrect answers on the same questions.

Depth boundaries. After establishing effective early transfer, we summarize the curves by their first half-effect boundaries:

$$\begin{aligned} T &= \min\{L : \rho(L) < 0.5\}, \\ R &= \min\{L : d(L) \le 0.5\}, \qquad \Delta = R - T. \end{aligned} \tag{5}$$

When both crossings exist, Δ measures their separation. The weak-transfer, strong-dependence set $\mathcal{W} = \{L : \rho(L) < 0.5,\ d(L) > 0.5\}$ can contain several intervals. Figure 3 retains later threshold crossings.

The execution state before generation. For one question, split the complete n-token prompt before its last token u_*. The processed prefix has positions $\mathcal{P} = \{0, \dots, n-2\}$, including the skill and question. Let $\overline{\mathcal{I}} = \mathcal{P} \setminus \mathcal{I}$. Attention reuses cached keys and values for these processed tokens [21]. At this point, define $C_X^\ell[\mathcal{I}] = (K_X^\ell[\mathcal{I}], V_X^\ell[\mathcal{I}])$, collect all layers as $C_X[\mathcal{I}]$, and write $\mathcal{B}_X = C_X[\overline{\mathcal{I}}]$ for the other prefix caches. With model parameters θ, positions, stopping rule and greedy decoding fixed, the continuation is

$$\hat{y}_X = \mathcal{G}_{\theta, M_X}(u_*, C_X[\mathcal{I}], \mathcal{B}_X). \tag{6}$$

Here M_X specifies attention masks, and $\mathcal{G}$ includes computing u_* and generating the answer from both source and other prefix caches.

Cache replacement and joint repair. After prefix prefill and before computing u_*, cache replacement writes

$$C: \quad C_X^\ell[\mathcal{I}] \leftarrow C_D^\ell[\mathcal{I}], \qquad 0 \le \ell \le L. \tag{7}$$

The prefix uses no state write for C ($X = R$), or a block-L write for SC ($X = S$). Cache replacement preserves other prefix entries, giving $\mathcal{B}_C = \mathcal{B}_R$ and $\mathcal{B}_{SC} = \mathcal{B}_S$. Indirect effects of S can change $\mathcal{B}_S$ relative to $\mathcal{B}_R$. Under the causal assumptions in Appendix B.2, ideal state replacement reconstructs source KV at layers above L. The explicit cache write supplies layers 0 through L, giving donor-aligned source caches under SC while retaining the receiver's other prefix caches $\mathcal{B}_S$. Figure 2 shows the write schedule.

Exact copying removes the replaced entries' contribution to squared cache distance from the donor. Appendix B.3 derives this identity and a sufficient condition for matching donor outputs under bounded changes in token scores.

Reading cuts and their timing. A *skill-reading cut* blocks reader-to-source attention. A *within-skill attention cut* blocks source-to-source attention. In *SC* + cut, the reading mask is

$$\widetilde{z}_{ij}^{\ell} = \begin{cases} -\infty, & \ell \geq L+1,\ i \in \mathcal{J},\ j \in \mathcal{I}, \\ z_{ij}^{\ell}, & \text{otherwise.} \end{cases} \tag{8}$$

It is installed before prefix prefill and persists during decoding. Source-to-source attention remains unmasked, but reader computation can change already during prefill. Consequently both M_X and $\mathcal{B}_X$ in Eq. (6) can differ from *SC*. The separate original-run path tests start their cuts at L, not $L+1$. Adding a within-skill cut to a persistent reading cut is structurally redundant for the output (Appendix B.1).

Repair contrasts. On a fixed evaluation set, we measure the cache gain $SC - S$ and the reading-cut loss $SC - (SC + \text{cut})$ in accuracy points. The interaction Γ compares the joint gain with the sum of the individual gains. Appendix B.1 gives their formulas.

4 Experiments

The experiments follow three questions. Does early state transfer carry skill-relevant information? Does that transfer weaken before dependence on the skill does? Can source-cache replacement restore part of the late transfer deficit?

4.1 Experimental setup

Tasks and models. We pair MedCalc-Bench [15] with skills from SRA-Bench, a benchmark that pairs questions with reusable skill documents [23], covering 1,100 questions from 55 clinical calculators. We also use SRA-Bench's TheoremQA subset, with 747 questions and 320 skills. TheoremQA tests the application of theorems across mathematics, physics, electrical engineering and computer science, and finance [5], extending the evaluation beyond clinical calculations. Each question receives its designated skill. Skill selection is held fixed.

The four main models are Qwen3-8B [30], Mistral-7B-Instruct-v0.3 [14], Llama-3.1-8B-Instruct [9] and DeepSeek-LLM-7B-Chat [7]. The depth study covers all eight combinations of model and benchmark (Table 2). Qwen3-8B and Mistral on TheoremQA use regenerated answers with independently checked grading (Appendix A.2). Models generate step-by-step answers without tools using greedy decoding, and the benchmark scorers evaluate those answers. On the full MedCalc benchmark, Qwen3-8B achieves 74.2% accuracy with the correct skill, 34.5% without one, and 31.5% with an unrelated skill.

Analysis populations. We select questions using earlier with-skill and without-skill outcomes, then re-evaluate unmodified runs, or baselines, under each protocol. For skill-level screening, we retain a skill only if the wrong-skill receiver answers none of its sampled questions correctly.

For Qwen3-8B/MedCalc, we use 97 questions for *L*2 validation and 135 for *L*8 controls (Table 1), 134 for common depth curves (Figure 3), 124 for the paired-outcome check (§4.3), and 114 for repair and path cuts (Figure 4). The control donor answers 124/135 correctly. The common depth set intersects transfer eligibility with correct-with-skill, incorrect-without-skill knockout outcomes. The paired check also requires correct donor answers under both protocols.

All *L*2 cells and the added *L*8 cells also require correct donor and incorrect no-skill baselines. Each block reports its own n/G, since their eligible sets can differ. Repair uses that rule plus an incorrect receiver for each item. Llama and DeepSeek reuse fixed candidate pools selected by earlier models. Intervention estimates are conditional on these selection rules. Each depth panel uses common question IDs for its two interventions. Repair and path tests likewise share question IDs within each model and task combination, using their own fresh-baseline screen. Appendix A details the distinct selection rules.

The complete-source study uses separate fixed historical pools totaling 580 model and question pairs and retains fresh baseline failures. It compares an unrelated-skill receiver (Wrong) with a fixed unrelated text matched to the source length (Neutral). All complete-source conditions use matched attention and decoding settings. Appendix B.4 gives the pools and protocol.

Protocol consistency. Historical Qwen3-8B and regenerated Qwen/Mistral TheoremQA runs use different attention implementations for transfer and knockout. Llama and DeepSeek use matching implementations. Repair and path cuts share a protocol with fresh baselines. Identity controls copy receiver states back to their original positions and reproduce its output tokens exactly. Resetting the TheoremQA evaluator before every answer and reversing grading order preserve the labels. Implementation details and corrections are in Appendix A.

4.2 Validation of skill-state transfer

Early transfer recovery. At layer 2, correct-skill states give 56.0% to 93.4% accuracy across all eight combinations, compared with 1.9% to 14.8% for permuted states and 1.8% to 14.8% for norm-matched noise (Table 1, upper block). Donors are correct and receivers incorrect on these screened sets, so the accuracies also give the percentage of donor gain recovered. This establishes effective early transfer before comparing its depth profile with skill dependence.

Content and position controls at a fixed depth. The lower block of Table 1 retains the *L*8 controls. For Qwen3-8B/MedCalc, skill-span transfer answers 110/135 questions correctly, against the donor's 124/135 and receiver's zero, recovering 0.89 of the donor's gain. Last-token and last-16-token writes have smaller effects but also replace fewer positions. The matched-prefix comparison copies equal numbers of leading skill positions and recovers 0.65 with correct content versus 0.05 with unrelated content. Permuting positions or injecting norm-matched noise gives little recovery. This noise preserves the Euclidean length of the skill-state change. Rescaling the correct patch to the receiver's norm still recovers 0.87. These controls support a role for the skill's content and arrangement. At *L*8, several other models have already crossed their transfer boundary (Figure 3).

Table 1: Early transfer at $L2$ and fixed-layer controls at $L8$. Entries are accuracy (%), with separate question/skill counts (n/G) for each block. $L2$ establishes effective early transfer across all eight combinations. $L8$ tests content and position controls at a fixed depth, which is already beyond some transfer boundaries. Sample selection is specified in Section 4.1.

Intervention	Qwen3-8B		Mistral-7B		Llama-3.1-8B		DeepSeek-7B	
	MedCalc	TheoremQA	MedCalc	TheoremQA	MedCalc	TheoremQA	MedCalc	TheoremQA
Early validation at L2								
n/G	97/11	61/51	35/13	61/51	112/20	92/71	25/13	54/45
Correct skill span	**89.7**	**93.4**	**68.6**	**85.2**	**90.2**	**76.1**	**56.0**	**57.4**
Correct span, permuted	2.1	14.8	5.7	11.5	6.2	8.7	12.0	1.9
Norm-matched noise	4.1	14.8	5.7	6.6	1.8	6.5	4.0	1.9
Fixed-layer controls at L8								
n/G	135/14	57/49	35/13	61/51	112/20	92/71	25/13	54/45
Correct skill donor	91.9	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Wrong skill receiver	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Correct skill span	81.5	78.9	20.0	37.7	33.0	23.9	8.0	11.1
Last prompt token	3.0	10.5	5.7	1.6	2.7	12.0	0.0	1.9
Last 16 prompt tokens	5.9	12.3	8.6	6.6	2.7	4.3	4.0	11.1
Correct matched prefix	59.3	71.9	17.1	37.7	35.7	22.8	12.0	7.4
Other-skill matched prefix	4.4	14.0	2.9	8.2	8.0	9.8	8.0	5.6
Correct span, permuted	4.4	22.8	14.3	14.8	4.5	14.1	4.0	5.6
Norm-matched noise	2.2	19.3	8.6	4.9	4.5	9.8	4.0	0.0
Identity patch	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Cross-question transfer. In the main prompt order, the skill precedes the question. Causal attention prevents those skill states from encoding a later question. We additionally place the skill after the question and take donor states from other questions using the same skill. For Qwen3-8B/MedCalc, these states recover 0.98 at layer 8 on 151 eligible items from 17 calculators, close to 1.02 for a donor answering the current question. At layer 16, the values separate to 0.29 and 0.49. Early transfer thus carries information shared across questions, with greater question sensitivity later.

4.3 Weak transfer with persistent source dependence

Depth separation. Early transfer is effective, but it weakens before source dependence does. Figure 3 shows the full curves, and Table 2 summarizes their boundaries.

The dependence boundary follows the transfer boundary in all eight comparisons. The gaps are 8 to 20 layers, or 22.2% to 62.5% of model depth (Table 2).

A representative depth profile. For Qwen3-8B/MedCalc, recovery drops from 0.73 at layer 12 to 0.16 at 13 and stays below 0.08 from 15 onward. Dependence remains 0.93 at 13 and 0.75 at 24. Throughout layers 13 to 24, transfer restores less than half of the donor's gain while knockout removes more than half of the originally correct answers (Figure 3).

Paired outcomes and robustness. The paired check confirms the separation on individual questions. For Qwen3-8B/MedCalc, both interventions yield incorrect answers on 100/124 (80.6%) questions at layer 13, and 113/124 (91.1%) at 16 (Appendix B). The boundary ordering also holds at thresholds 0.4 and 0.6 in all eight comparisons. Broader screening and later threshold crossings appear in Appendix B.

4.4 Partial restoration with source-cache replacement

Testing an omitted source component. Figure 4 compares all eight model and task combinations, separating receiver repair at $L16$ (panel a) from attention cuts in the original correct-skill run at $L8$, $L16$ and $L24$ (panel b).

Repair gains in the receiver. Adding caches improves accuracy over state transfer alone in all eight layer-16 comparisons, by 6.8 to 43.4 percentage points (Figure 4a). Joint-repair accuracy ranges from 10.2% to 62.3%. For Qwen3-8B/MedCalc, S, C and SC answer 4, 8 and 48 of 114 questions correctly (3.5%, 7.0%, 42.1%). The cache benefit is $\Delta_{\text{cache}} = (48 - 4)/114$, or 38.6 percentage points. The accuracy interaction is $\Gamma = (48-4-8+0)/114$, or 31.6 points (Eq. (9)). At layer 24, SC reaches 25.4%, versus 0.9% for S and 18.4% for C. The added benefit of states alongside caches varies. For Mistral/MedCalc at $L16$, SC and C both reach 18.4%, with $\Gamma = -7.9$ points. Table 4 reports all individual conditions and contrasts.

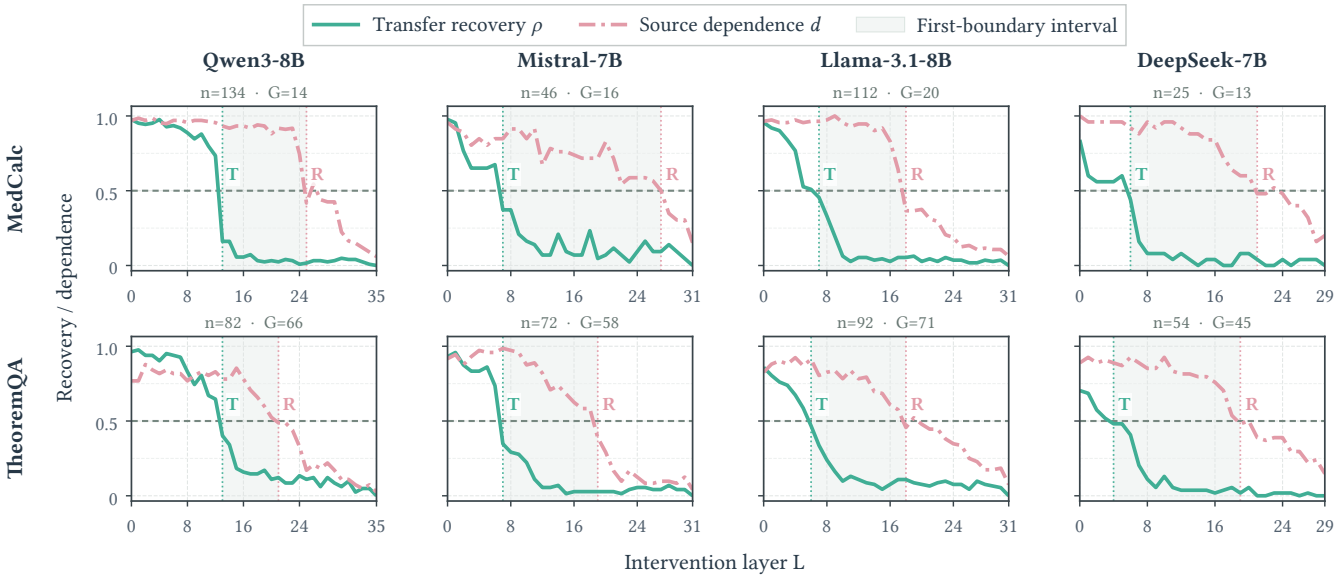


Figure 3: State transfer weakens before source-attention dependence across eight combinations of model and task. Rows show MedCalc (top) and TheoremQA (bottom), with models aligned by column. Mint: recovery $\rho(L)$ in the wrong-skill receiver. Pink dash-dotted: dependence $d(L)$ in the original correct-skill run. Each panel uses common eligible questions with each protocol's own baselines. G counts skills. The horizontal line marks 0.5 and mint/pink vertical markers locate the first boundaries T/R. Shading spans their interval (Table 2). Later threshold crossings remain visible. Knockout blocks both skill reading and within-skill attention. Every layer is measured.

Table 2: Main depth comparison across four open-weight models and two benchmarks. n/G gives common eligible questions/skill groups. T and R are the first half-effect crossings for transfer ($\rho < 0.5$) and aggregate source dependence ($d \le 0.5$). $\Delta = R - T$ is in layers. Δ/D normalizes the gap by the model's layer count. All eight cells use audited common-item populations.

Model	MedCalc-Bench					TheoremQA				
	n/G	T	R	Δ	Δ/D (%)	n/G	T	R	Δ	Δ/D (%)
Qwen3-8B	134/14	13	25	**12**	**33.3**	82/66	13	21	**8**	**22.2**
Mistral-7B	46/16	7	27	**20**	**62.5**	72/58	7	19	**12**	**37.5**
Llama-3.1-8B	112/20	7	18	**11**	**34.4**	92/71	6	18	**12**	**37.5**
DeepSeek-7B	25/13	6	21	**15**	**50.0**	54/45	4	19	**15**	**50.0**

Reading under joint repair. Reading cuts reduce joint-repair accuracy by 2.2 to 37.7 points across all eight layer-16 comparisons. For Qwen3-8B/MedCalc, accuracy falls from 42.1% to 7.0%, giving $\Delta_{\text{read}} = (48 - 8)/114$, or 35.1 points. Thus the cross-model improvement from adding caches coexists with continued use of the skill. The Qwen3-8B/MedCalc result at $L24$ differs: the cut gives 27.2% versus 25.4% without it. Appendix B.1 documents the layer grid and checks sensitivity to outputs stopped at the generation limit.

Attention paths in the original run. Figure 4(b) tests the original correct-skill execution. At $L8$, both types of cut cause large losses across the eight rows. At $L24$, Skill reading loses 11.9% to 74.6% of correct answers, exceeding Within-skill attention by 6.8 to 71.1 points in every row. The Qwen3-8B/MedCalc contrast is 74.6% versus 3.5%. At this measured depth, later positions' access to the skill has a larger effect than attention among skill positions across all eight settings.

Limits of recovery with complete source KV. The final controls test what limits recovery after all source KV has been supplied. Table 3 separates the skill's cached information from the receiving execution's earlier processing of the prompt. C_{all} copies donor keys and values at every layer for the skill positions, leaving caches at other positions unchanged. Those other caches can retain the effects of processing the unrelated skill before the copy. Full copies the complete prefix cache, including positions outside the skill. Replay instead copies all source KV and then processes the later prompt tokens again, so their caches are recomputed while the correct skill's cached information is available. The B condition provides the complementary comparison by copying only the caches outside the

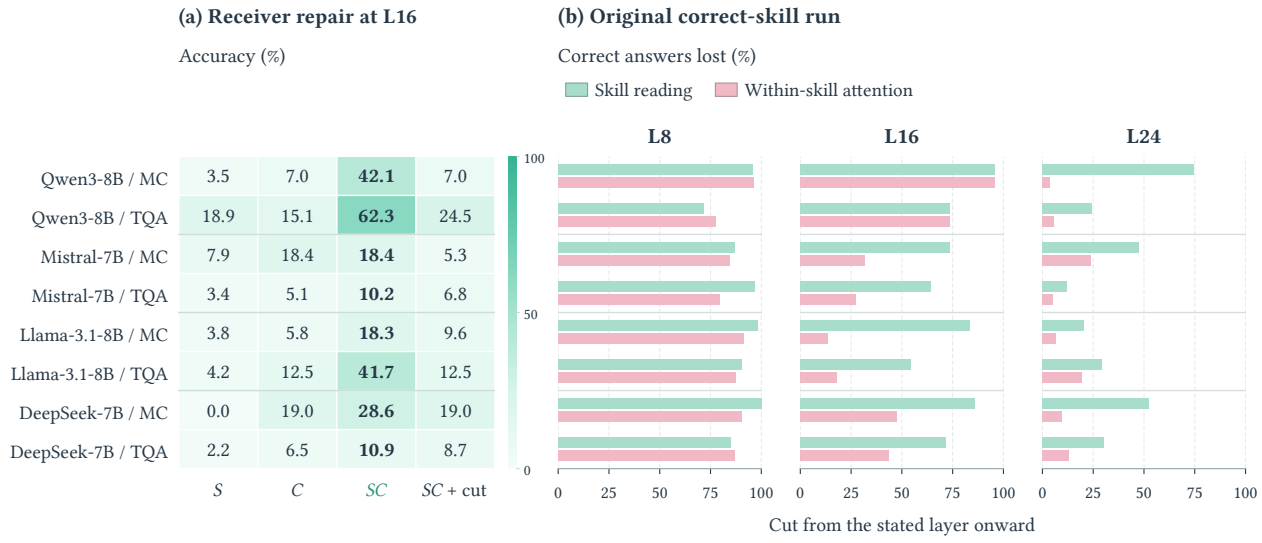


Figure 4: Repair and attention cuts across four models and two benchmarks. MC denotes MedCalc and TQA denotes TheoremQA. (a) Receiver accuracy at $L16$ for state replacement (S), cache replacement (C), joint repair (SC), and joint repair with a reading cut from $L17$ onward (SC + cut). (b) Correct answers lost in the original correct-skill run under cuts from $L8$, $L16$ or $L24$ onward during prefill and decoding. Skill reading blocks reader-to-source attention. Within-skill attention blocks source-to-source attention. Each row uses the same questions in both panels, with counts in Table 4. Higher values mean better recovery in (a) and greater disruption in (b). Layer numbers index each model's blocks.

skill positions. These controls use a donor computed for the same question. Comparing C_{all} with Full tests the effect of also supplying donor caches outside the skill. Comparing it with Replay tests whether recomputing the later prompt can recover performance while retaining the receiving prompt.

Across all eight combinations, every source-position K/V tensor matches the donor under SC at $L8$, $L16$ and $L24$ before continuation. An additional explicit all-layer write changes none of the 3,480 paired generated sequences. Copying all source KV without state replacement (C_{all}) reaches 9.3% to 58.7% in Wrong and 17.4% to 73.0% in Neutral, below each cell's donor (85.2% to 97.6%, Table 3).

Copying the complete prefix KV reproduces donor tokens in all 1,160 receiver and question pairs. Copying all source KV and recomputing the later prompt, denoted Replay, reaches within 4.2 accuracy points of the donor in every cell, while allowing different generated sequences. These controls identify receiving state outside the skill positions as a factor in recovery after source KV is complete.

Accuracy and agreement with the donor's generated sequence describe different aspects of this recovery. Accuracy measures whether the answer satisfies the benchmark's scoring rule. Sequence agreement requires every generated token to match, including the intermediate explanation. Full meets this stronger condition on every tested pair, reproducing the donor's correct answers and its errors. Replay achieves similar task performance while often following a different generated sequence (Table 5). For skill reuse, these comparisons distinguish preserving task performance from reproducing a particular execution.

The depth effect of joint repair also varies. SC@8 exceeds SC@24 in both contexts in six of eight combinations. The direction depends on the receiving context for Llama/TheoremQA and DeepSeek/MedCalc. Although source KV is complete in these conditions, a state write at a different layer changes the remaining computation during prefill. The later prompt positions can consequently reach generation with different caches. The depth comparison thus concerns how the receiving execution uses the transferred states as well as which source entries are supplied. Appendix B.4 gives the paired differences and additional content and layer-window controls.

5 Discussion

Potential applications. Our findings suggest a way to evaluate the reuse of contextual skills. Consider a Web agent that follows the same instructions to extract specified information from different pages. The instructions stay the same, while the page content and the user's request change. Reusing the states formed from those instructions would need to preserve the agent's ability to answer the new request. The separation observed here motivates testing that ability directly, alongside the benefit of providing the original instructions for each request.

An evaluation can first compare a receiving execution that uses the stored states with an execution that processes the correct skill for that request. This measures how much task performance survives reuse. It can then hold the copied skill information fixed and compare retaining the receiving execution's existing caches with recomputing the later prompt. That comparison checks whether the copied information works with the history already stored by

Table 3: Recovery with complete source KV and receiving-history controls. Entries are accuracy (%). C_{all} copies all source KV, *SC@L* adds a state write at layer L, and B copies only other prefix KV. Full copies the complete prefix and equals the donor. Replay copies all source KV and recomputes the later prompt. Wrong and Neutral are defined in Section 4.1, with separate sample counts in Table 5.

Model / task	Context	Base	C_{all}	*SC@8*	*SC@16*	*SC@24*	B	Full (= donor)	Replay
Qwen3-8B / MC	Wrong	4.8	29.8	91.9	41.9	25.8	2.4	97.6	96.8
	Neutral	4.0	25.0	91.1	28.2	26.6	8.1	97.6	96.8
Qwen3-8B / TQA	Wrong	6.3	58.7	95.2	60.3	50.8	20.6	96.8	98.4
	Neutral	25.4	73.0	84.1	65.1	71.4	34.9	96.8	98.4
Mistral-7B / MC	Wrong	6.5	15.2	56.5	15.2	13.0	4.3	91.3	89.1
	Neutral	6.5	17.4	39.1	17.4	17.4	19.6	91.3	89.1
Mistral-7B / TQA	Wrong	3.1	10.8	49.2	9.2	13.8	24.6	93.8	93.8
	Neutral	4.6	23.1	43.1	21.5	21.5	27.7	93.8	93.8
Llama-3.1-8B / MC	Wrong	1.8	16.1	73.2	17.0	16.1	1.8	96.4	96.4
	Neutral	8.0	27.7	48.2	25.9	25.9	4.5	96.4	96.4
Llama-3.1-8B / TQA	Wrong	5.4	43.5	45.7	46.7	51.1	10.9	90.2	89.1
	Neutral	10.9	46.7	57.6	57.6	51.1	16.3	90.2	89.1
DeepSeek-7B / MC	Wrong	0.0	29.2	41.7	29.2	33.3	20.8	91.7	87.5
	Neutral	8.3	41.7	25.0	29.2	33.3	20.8	91.7	87.5
DeepSeek-7B / TQA	Wrong	0.0	9.3	22.2	9.3	9.3	14.8	85.2	83.3
	Neutral	14.8	31.5	44.4	31.5	29.6	22.2	85.2	83.3

the receiving execution. The complete-source controls motivate this second check: matching all skill-position KV still leaves performance sensitive to the state of other positions. When task inputs change, the amount of prompt processing that can be reused therefore needs to be assessed together with the skill representation.

Continued access to the skill is a further part of the evaluation. A method that preserves accuracy after copying states may still require attention to the skill positions. Comparing runs with and without this access distinguishes reuse that depends on continued skill reading from reuse that does not. This distinction matters when the intended benefit is to reduce repeated instruction processing. Studies in interactive Web settings can evaluate task performance, latency and memory use together, counting both the copied cache and any later prompt computation required to make use of it.

Future work. A next step is to investigate which questions benefit from cache replacement. The mathematical analysis connects remaining cache differences to output stability under explicit assumptions. Measuring these differences, their influence on token predictions and the donor's prediction margins could test whether this framework helps distinguish successful from unsuccessful repairs. The complete-source controls further motivate testing which parts of the receiving history must be recomputed, and whether the required budget can be predicted on held-out skills. The heterogeneous depth effects suggest that a recomputation rule should be tested across model/task cells rather than inferred from one depth curve. Separating attention cuts during prompt processing from those during answer generation would clarify when continued skill access matters. Finally, multi-step Web workflows would assess how far the observed separation extends beyond the question-answering settings studied here.

6 Conclusion

Using a contextual skill and reusing its internal states are distinct capabilities. Across four models and two benchmarks, state transfer weakens before the original execution's dependence on skill attention, with half-effect boundaries separated by 8 to 20 layers. Early content controls establish that the transferred states carry skill-relevant information, and the depth comparisons show that this information becomes harder to reuse while the skill remains useful.

Source-cache replacement provides a way to recover part of that transfer. At layer 16, adding caches improves accuracy over states alone by 6.8 to 43.4 percentage points in all eight comparisons, while cutting subsequent skill reading reduces the joint-repair accuracy in each. Complete-source controls further show that skill-position KV alone is insufficient to recover donor accuracy. Full-prefix copying reproduces the donor's generated sequence, while the depth effect of joint repair varies across models and tasks. The mathematical analysis connects cache alignment to output stability under explicit conditions. Together, these findings motivate evaluating state reuse separately from continued skill use and including cached information in the design of reusable skill representations.

References

[1] Thibaud Ardoin, Semira Einsele, Evis Bregu, and Gerhard Wunder. 2026. Prompt Compression via Activation Aggregation. *arXiv preprint arXiv:2607.08399* (2026).

[2] Irina Bigoulaeva, Jonas Rohweder, Subhabrata Dutta, and Iryna Gurevych. 2026. Patches of Nonlinearity: Instruction Vectors in Large Language Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 12209–12262. doi:10.18653/v1/2026.acl-long.559

[3] Lucas Caccia, Alan Ansell, Edoardo Ponti, Ivan Vulić, and Alessandro Sordoni. 2025. Training Plug-n-Play Knowledge Modules with Deep Context Distillation. In *Conference on Language Modeling (COLM)*. arXiv:2503.08727.

[4] Rujikorn Charakorn, Edoardo Cetin, Shinnosuke Uesaka, and Robert Tjarko Lange. 2026. Doc-to-LoRA: Learning to Instantly Internalize Contexts. *arXiv preprint arXiv:2602.15902* (2026).

[5] Wenhu Chen, Ming Yin, Max Ku, Pan Lu, Yixin Wan, Xueguang Ma, Jianyu Xu, Xinyi Wang, and Tony Xia. 2023. TheoremQA: A Theorem-driven Question Answering Dataset. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. https://arxiv.org/abs/2305.12524 arXiv:2305.12524.

[6] Guy Davidson, Todd M. Gureckis, Brenden M. Lake, and Adina Williams. 2025. Do different prompting methods yield a common task representation in language models?. In *Advances in Neural Information Processing Systems*, Vol. 38. doi:10.52202/085713-2380

[7] DeepSeek-AI. 2024. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. *arXiv preprint arXiv:2401.02954* (2024).

[8] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. In-context Autoencoder for Context Compression in a Large Language Model. In *International Conference on Learning Representations (ICLR)*. arXiv:2307.06945.

[9] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).

[10] Sina Hajimiri, Masih Aminbeidokhti, Jose Dolz, Ismail Ben Ayed, Issam H. Laradji, Spandana Gella, and Nicolas Gontier. 2026. Are Online Skill and Memory Modules Always Worth Their Tokens? A Budget-Constrained Study of Web Agents. *arXiv preprint arXiv:2606.15017* (2026). https://arxiv.org/abs/2606.15017

[11] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? *arXiv preprint arXiv:2603.15401* (2026).

[12] Stefan Heimersheim and Neel Nanda. 2024. How to use and interpret activation patching. *arXiv preprint arXiv:2404.15255* (2024).

[13] Roee Hendel, Mor Geva, and Amir Globerson. 2023. In-Context Learning Creates Task Vectors. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. arXiv:2310.15916.

[14] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).

[15] Nikhil Khandekar, Qiao Jin, Guangzhi Xiong, Soren Dunn, Serina S. Applebaum, Zain Anwar, Maame Sarfo-Gyamfi, Conrad W. Safranek, Abid A. Anwar, Andrew Zhang, Aidan Gilson, Maxwell B. Singer, Amisha Dave, Andrew Taylor, Aidong Zhang, Qingyu Chen, and Zhiyong Lu. 2024. MedCalc-Bench: Evaluating Large Language Models for Medical Calculations. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. arXiv:2406.12036.

[16] Bojie Li. 2026. Models Take Notes at Prefill: KV Cache Can Be Editable and Composable. *arXiv preprint arXiv:2606.17107* (2026). https://arxiv.org/abs/2606.17107

[17] Xiangyi Li, Yimin Liu, Wenbo Chen, et al. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670* (2026).

[18] Yuxuan Li, Declan Campbell, Stephanie C. Y. Chan, and Andrew Kyle Lampinen. 2025. Just-in-time and distributed task representations in language models. *arXiv preprint arXiv:2509.04466* (2025).

[19] Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2023. Learning to Compress Prompts with Gist Tokens. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2304.08467.

[20] Aditya Pola and Vineeth N. Balasubramanian. 2025. Where does an LLM begin computing an instruction?. In *UniReps Workshop*. extended abstract. arXiv:2511.10694.

[21] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. *arXiv preprint arXiv:1911.02150* (2019). https://arxiv.org/abs/1911.02150

[22] Suzanna Sia, David Mueller, and Kevin Duh. 2024. Where does In-context Learning Happen in Large Language Models?. In *Advances in Neural Information Processing Systems (NeurIPS)*. doi:10.52202/079017-1030

[23] Weihang Su, Jianming Long, Qingyao Ai, Qiaozhi He, Yichen Tang, Changyue Wang, Yiteng Tu, Yingbo Wang, and Yiqun Liu. 2026. Skill Retrieval Augmentation for Agentic AI. *arXiv preprint arXiv:2604.24594* (2026).

[24] Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. 2024. Function Vectors in Large Language Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.15213.

[25] Yusuke Tsuzuku, Issei Sato, and Masashi Sugiyama. 2018. Lipschitz-Margin Training: Scalable Certification of Perturbation Invariance for Deep Neural Networks. In *Advances in Neural Information Processing Systems*, Vol. 31. https://arxiv.org/abs/1802.04034

[26] Sankaran Vaidyanathan, David Arbour, Aaron Mueller, Scott Niekum, and David Jensen. 2026. The Curse of Multiple Mediators: Hidden Interaction Effects in Activation Patching. *arXiv preprint arXiv:2606.27510* (2026). https://arxiv.org/abs/2606.27510

[27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, Vol. 30. https://arxiv.org/abs/1706.03762

[28] Zhaoyang Wang, Qianhui Wu, Xuchao Zhang, Chaoyun Zhang, Wenlin Yao, Fazle Elahi Faisal, Baolin Peng, Si Qin, Suman Nath, Qingwei Lin, Chetan Bansal, Dongmei Zhang, Saravan Rajmohan, Jianfeng Gao, and Huaxiu Yao. 2026. WebXSkill: Skill Learning for Autonomous Web Agents. *arXiv preprint arXiv:2604.13318* (2026). https://arxiv.org/abs/2604.13318

[29] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 63897–63911. https://proceedings.mlr.press/v267/wang25bx.html

[30] An Yang, Anfeng Li, et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).

[31] Zhipeng Yang, Junzhuo Li, Siyu Xia, and Xuming Hu. 2025. Internal Chain-of-Thought: Empirical Evidence for Layer-wise Subtask Scheduling in LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 22536–22564. doi:10.18653/v1/2025.emnlp-main.1147

[32] Barry Zhang, Keith Lazuka, and Mahesh Murag. 2025. Equipping agents for the real world with Agent Skills. Anthropic Engineering. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills

[33] Fred Zhang and Neel Nanda. 2024. Towards Best Practices of Activation Patching in Language Models: Metrics and Methods. In *International Conference on Learning Representations (ICLR)*. arXiv:2309.16042.

A Experimental protocols and provenance

Tasks and scoring. MedCalc has 20 questions for each of 55 SRA-Bench calculators, scored with prescribed numerical tolerances. TheoremQA uses released prompts, skills and grading. Groups denote calculators or TheoremQA skills. Skills are supplied directly, without tools.

Evaluation populations. The corrected Qwen3-8B/MedCalc pool contains 467/470 historically rescued items from 48 calculators. Whole-group screening retains 135/14 items/calculators, including donor failures. Correct counts are 124 for donors and 110 for layer-8 span patches. The common depth set has 134 items with separate intervention baselines. Section 4.1 distinguishes these populations from the added controls, item-screened repair sets and fixed complete-source pools. The common DeepSeek/MedCalc set has 25 items. Threshold sensitivity does not test sample-composition robustness.

A.1 Implementation details

Wrong-skill selection and token matching. MedCalc's fixed distractor is APACHE II, a clinical severity score (`medcalcbench_043`) outside the selected calculators. TheoremQA uses `theoremqa_148`, on representation theorems and differential equations, selected by fewest assigned questions then greatest length. Its assigned items are excluded. We repeat distractor tokens as needed and copy the first m into the m-token source span, without padding or changing other tokens and positions.

Prompt and source boundaries. The user turn orders `Relevant Skill:`, skill text, a blank line and the question. MedCalc requests calculations ending with `ANSWER: <your answer>`, under a system instruction to calculate a score step by step from a patient note. TheoremQA retains its science-teacher system message,

`Problem:<question>` and `Solution:` format, and "Therefore, the answer is ..." ending. Tokenization applies the chat template's role labels and generation prompt. Character offsets locate the skill tokens, excluding header and separator. We replace IDs in [lo, hi), including the start but excluding the end, without retokenization.

Models and decoding. Qwen3-8B has 36 layers and width 4096, Mistral-7B has 32 layers and width 4096, Llama-3.1-8B has 32 layers, and DeepSeek-7B has 30. Transfer/knockout use greedy decoding with 900 new tokens and Qwen's extended reasoning disabled. Llama and DeepSeek use BF16 (16-bit floating point) and eager attention (explicit attention scores). Regenerated Qwen/Mistral TheoremQA uses PyTorch scaled dot-product attention (SDPA) for transfer and eager attention for knockout. DeepSeek/Mistral merge system instructions into the user turn. Llama retains original weights, configuration and vocabulary with the public distribution's chat template. Repair/path runs use BF16, eager attention and 1,536 new tokens, splitting prefill before the final prompt token for cache writes. Cuts span prefill and decoding. Layer indices denote blocks, not matched computational stages across models.

Position and state checks. Length matching preserves question positions. Patches target prefill block outputs, separate from scored answer tokens. Identity controls restore receiver states, and final-layer patches reproduce donor next-token scores. Cuts preserve causality. Split-prefill checks on one item from each of 14 calculators reproduce tokens in all 84 receiver-state tests. Splitting changes one donor's and one receiver's correctness, motivating fresh matched repair baselines.

A.2 Evaluation scope and intervention provenance

Attention-mask correction. Capture and decoding masks match in all main state interventions. An earlier omission failed identity checks. Correction restores exact agreement, including all 467 MedCalc items. Knockout has no donor capture.

Extension pools and grading checks. Llama/DeepSeek rescreen frozen pools of 446 MedCalc and 255 TheoremQA items from earlier eligible transfer sets. One DeepSeek/MedCalc prompt cannot reserve 900 answer tokens and is excluded before inference. Qwen/Mistral TheoremQA uses 168/142 candidates, yielding 82/66 and 72/58 common questions/skills. Official grading resets parser state per answer. Reverse-order and separate-process checks agree before eligibility is recomputed.

Layer-2 controls use their own screened sets. In the model order of Table 1, the MedCalc question counts are 97, 35, 112, 25 and correct-span counts are 87, 24, 101, 14. TheoremQA counts are 61, 61, 92, 54, with 57, 52, 70, 31 correct. The DeepSeek/TheoremQA cells use the final 54-item grading correction.

Artifact availability. Stored artifacts contain configurations, question IDs, answers and token sequences. An anonymous review archive is not yet linked.

B Additional transfer and source-dependence analyses

On the 467 corrected Qwen3-8B/MedCalc questions from 48 calculators, group/item receiver screens retain 135/14 and 369/45 questions/calculators, respectively.

Broader screening and boundary sensitivity. The item-level screen retains the transfer decline. Boundary ordering also holds on common questions at transfer thresholds $\tau = 0.4/0.6$ and dependence thresholds $1 - \tau$. Qwen3-8B/MedCalc dependence first falls to at most half at layer 25, exceeds half at 26, and stays below half from 27 onward. The curves retain these recrossings.

Same-question joint failures. With correct donors under both protocols and incorrect transfer-receiver and no-skill knockout baselines, joint failures are 100/124 (80.6%) for Qwen3-8B at $T = 13$ and 25/43 (58.1%) for Mistral-7B at $T = 7$, on 14/15 MedCalc groups. These sets differ from Table 2. Four items in each pass the transfer no-skill baseline despite failing it under knockout.

B.1 Source-cache repair and attention-path experiments

Repair/path tests share fresh donor-correct, receiver-incorrect and no-skill-incorrect items within each row of Table 4. Qwen3-8B/MedCalc retains 114/124 candidates from 14 calculators (116 pass the first two criteria). All 372 baseline generations match across suites, and all 48 receiver-state writebacks are token-exact. Repair layers are 8/12/16/24, path-cut layers 8/16/24, with writes, masks and contrasts in Eqs. (7), (8) and (9).

Three repair contrasts. On one fixed evaluation set under one protocol, let $a_X(L)$ denote accuracy for condition X. Suppressing L on the right, define

$$\begin{aligned} \Delta_{\text{cache}}(L) &= a_{SC} - a_S, \\ \Gamma(L) &= (a_{SC} - a_S) - (a_C - a_R), \\ \Delta_{\text{read}}(L) &= a_{SC} - a_{SC+\text{cut}}. \end{aligned} \tag{9}$$

These accuracy contrasts measure cache benefit, interaction and reading dependence. Positive Γ means the joint gain exceeds the sum of individual gains. Multiply by 100 for percentage points.

Qwen3-8B/MedCalc truncation sensitivity. All outcomes are retained. At $L16$, S, SC and SC + cut truncate 4, 4 and 3 of 114 outputs, respectively. The $SC-S$ gain is 38.6 points, 38.5 on 96 items with untruncated baselines, and at least 35.1 under adversarial scoring of truncated S/SC answers. At $L24$, $SC/(SC + \text{cut})$ truncate 3/28 outputs without an additional measured accuracy loss.

At $L24$, reader/source-self cuts retain 29/114 and 110/114 correct answers, a 71.1-point gap. Counting all 21 truncated reader-cut outputs as correct gives 50/114, still below 110/114 for source-self cuts, which have no truncations. Both contrasts survive this reassignment.

Redundant combined cut. Under causal attention and position-wise operations, earlier tokens cannot read the skill. Persistent reading cuts prevent altered skill states from reaching later tokens. Induction over layers and generated tokens then makes the combined cut redundant. This requires blocking all later reading.

Table 4: Layer-16 repair accuracy (%) and contrasts (percentage points). n/G counts questions/skills passing fresh baseline checks. Each row shares questions across conditions, with donor accuracy 100% and receiver accuracy 0%. The three contrasts follow Eq. (9). Populations differ from Table 2.

Model	Task	n/G	S	C	SC	SC + cut	Δ_{cache}	Γ	Δ_{read}
Qwen3-8B	MedCalc	114/14	3.5	7.0	**42.1**	7.0	+38.6	+31.6	+35.1
Qwen3-8B	TheoremQA	53/43	18.9	15.1	**62.3**	24.5	+43.4	+28.3	+37.7
Mistral-7B	MedCalc	38/15	7.9	18.4	**18.4**	5.3	+10.5	−7.9	+13.2
Mistral-7B	TheoremQA	59/49	3.4	5.1	**10.2**	6.8	+6.8	+1.7	+3.4
Llama-3.1-8B	MedCalc	104/20	3.8	5.8	**18.3**	9.6	+14.4	+8.7	+8.7
Llama-3.1-8B	TheoremQA	72/56	4.2	12.5	**41.7**	12.5	+37.5	+25.0	+29.2
DeepSeek-7B	MedCalc	21/10	0.0	19.0	**28.6**	19.0	+28.6	+9.5	+9.5
DeepSeek-7B	TheoremQA	46/40	2.2	6.5	**10.9**	8.7	+8.7	+2.2	+2.2

Table 5: Complete-source baselines (accuracy, %) and sequence checks. n/G counts questions/skills. Replay equality counts match across Wrong/Neutral. Full matches all $2n$ donor sequences per row. Explicit all-layer rewriting preserves all $6n$ SC sequences.

Model / task	n/G	Donor	No skill	Full copy	Replay tokens equal
Qwen3-8B / MC	124/14	97.6	3.2	97.6	48/124
Qwen3-8B / TQA	63/53	96.8	7.9	96.8	33/63
Mistral-7B / MC	46/16	91.3	2.2	91.3	28/46
Mistral-7B / TQA	65/55	93.8	0.0	93.8	53/65
Llama-3.1-8B / MC	112/20	96.4	1.8	96.4	55/112
Llama-3.1-8B / TQA	92/71	90.2	6.5	90.2	42/92
DeepSeek-7B / MC	24/12	91.7	4.2	91.7	12/24
DeepSeek-7B / TQA	54/45	85.2	0.0	85.2	34/54

Table 6: Paired $SC@8 - SC@24$ accuracy differences (points) with complete source KV. Positive values favor $L8$. Intervals are exploratory 95% skill-group bootstrap intervals, unadjusted for multiple comparisons. Cap columns count generation-limit outputs at $L8/L24$.

Model / task	Wrong: gap [interval]	Neutral: gap [interval]	Wrong cap	Neutral cap
Qwen3-8B / MC	+66.1 [49.3, 78.5]	+64.5 [40.5, 82.8]	0/3	0/0
Qwen3-8B / TQA	+44.4 [32.2, 56.7]	+12.7 [0.0, 24.6]	0/0	0/1
Mistral-7B / MC	+43.5 [29.4, 61.1]	+21.7 [9.1, 35.3]	0/0	0/0
Mistral-7B / TQA	+35.4 [20.3, 50.0]	+21.5 [10.3, 33.3]	0/5	2/1
Llama-3.1-8B / MC	+57.1 [38.1, 72.5]	+22.3 [13.3, 29.5]	4/12	12/12
Llama-3.1-8B / TQA	−5.4 [−17.2, 7.2]	+6.5 [−4.8, 19.1]	8/3	4/5
DeepSeek-7B / MC	+8.3 [−6.9, 27.8]	−8.3 [−25.0, 16.7]	0/0	0/0
DeepSeek-7B / TQA	+13.0 [5.2, 22.6]	+14.8 [3.5, 26.9]	9/6	1/0

B.2 Cache coverage under causal prefill

This deduction assumes a deterministic causal Transformer with masked attention as its only cross-position operation [27], identical parameters and position IDs, and the same exact-arithmetic computation without extra attention cuts. Donor and receiver tokens differ only within the complete, contiguous source interval $\mathcal{I} = [\text{lo}, \text{hi})$ in the prefix.

Proposition (source-cache coverage). Suppose S replaces the entire source output of block L before the next block executes, and the cache write occurs only after prefix prefill. At the observation point

in Eq. (6),

$$C_S^\ell[\mathcal{I}] = \begin{cases} C_R^\ell[\mathcal{I}], & \ell \le L, \\ C_D^\ell[\mathcal{I}], & \ell > L. \end{cases} \tag{10}$$

Together with the explicit write at layers 0 through L, this gives $C_{SC}[\mathcal{I}] = C_D[\mathcal{I}]$.

Proof. Block-L KV is computed before the output patch, leaving source caches at $\ell \le L$ unchanged. Before lo, identical tokens and causality give identical states at every layer. After S, all block-L outputs at $i <$ hi therefore match the donor. At these positions, the next block reads only matching earlier or current states, producing matching source KV and outputs. Induction proves the upper-layer case. Equation (7) supplies the lower layers for SC. □

Remaining execution state. The post-prefill write preserves other-position caches: S and SC retain $\mathcal{B}_S$, while C retains $\mathcal{B}_R$. Thus,

$$\begin{aligned} \hat{y}_{SC} &= \mathcal{G}_{\theta,M_0}(u_*, C_D[\mathcal{I}], \mathcal{B}_S), \\ \hat{y}_D &= \mathcal{G}_{\theta,M_0}(u_*, C_D[\mathcal{I}], \mathcal{B}_D), \end{aligned} \tag{11}$$

where M_0 is the original mask. Since $\mathcal{B}_S$ need not equal $\mathcal{B}_D$, answers may differ. All-layer source-KV-only replacement instead preserves $\mathcal{B}_R$ and is generally not equivalent to SC.

Under SC + cut, source queries remain unmasked, so the same induction applies. However, $\mathcal{B}_{SC+\text{cut}}$ may differ from $\mathcal{B}_S$, and continuation uses a different mask. The contrast includes both prefill and decoding effects.

Code checks confirm write ordering. Elementwise checks verify BF16 coverage (Section 4.4), while the proposition assumes exact arithmetic.

B.3 A sufficient condition for behavioral recovery

After prefix prefill, flatten caches as $\mathbf{z}_X = (\mathbf{c}_X, \mathbf{b}_X)$. Here $\mathbf{c}_X$ contains source KV at layers $0, \dots, L$, and $\mathbf{b}_X$ contains higher-layer source KV and $\mathcal{B}_X$. The cache write gives $\mathbf{z}_{SC} = (\mathbf{c}_D, \mathbf{b}_S)$, preserving $\mathbf{b}_S$. For $\varepsilon_X = \|\mathbf{z}_X - \mathbf{z}_D\|_2$,

$$\begin{aligned} \varepsilon_S^2 &= \|\mathbf{c}_S - \mathbf{c}_D\|_2^2 + \|\mathbf{b}_S - \mathbf{b}_D\|_2^2, \\ \varepsilon_{SC}^2 &= \|\mathbf{b}_S - \mathbf{b}_D\|_2^2, \\ \varepsilon_S^2 - \varepsilon_{SC}^2 &= \|\mathbf{c}_S - \mathbf{c}_D\|_2^2 \ge 0. \end{aligned} \tag{12}$$

Here $\|\cdot\|_2$ is Euclidean length. Exact copying removes the replaced entries' squared error from the cache distance, independently of causal reconstruction.

Output stability. Fix the model, positions, last prompt token and mask. Let $F_t^D(\mathbf{z})$ map initial caches to token-t logits (unnormalized token scores) after donor history $y_{D,<t}$ and its cache updates. For the compared states, assume

$$\|F_t^D(\mathbf{z}) - F_t^D(\mathbf{z}')\|_\infty \le \kappa_t \|\mathbf{z} - \mathbf{z}'\|_2. \tag{13}$$

The infinity norm bounds the largest absolute logit change. Let $m_{D,t} > 0$ be the donor's highest logit minus its second-highest. Margin stability [25] gives

$$[\ \forall t \le N :\ 2\kappa_t \varepsilon_{SC} < m_{D,t}\ ] \implies \hat{y}_{SC} = \hat{y}_D, \tag{14}$$

where N includes the stopping token and ε_{SC} is defined in Eq. (12).

Proof. Each logit changes by at most $\kappa_t \varepsilon_{SC}$, so the donor's winning logit remains ahead by at least $m_{D,t} - 2\kappa_t \varepsilon_{SC} > 0$. Induction over the shared greedy history, including the common stopping rule, proves sequence equality. □

Different sequences may also answer correctly. A common sensitivity bound gives SC no larger error bound than S. Errors from different cache components can offset one another, so replacing one component can remove that offset and increase actual logit error. Accuracy need not improve. Sensitivities and margins remain unverified. The guarantee excludes SC + cut, which changes the mask.

B.4 Complete-source and receiving-history controls

Population and protocol. The 580 model/question pairs retain fresh baseline errors (Table 5). Qwen3-8B/MedCalc reuses 124 frozen candidates from its 49-condition study. The seven other pools use at most 124 IDs in original order. One DeepSeek/MedCalc item fails input-length preflight, leaving 456 new pairs. These pools precede repair outcomes and differ from Table 4's screened sets. They are neither full-benchmark samples nor independent questions across models. Wrong uses the original distractor. Neutral length-matches a fixed unrelated document by repetition or truncation. It may still influence answers. All conditions use the repair settings and evaluator checks in Appendix A.1. Layers 8/16/24 are block indices.

Conditions and numerical identities. The 24 conditions are four baselines (donor, Wrong, Neutral, no skill), SC/SC_{exact} at $L8/L16/L24$ in both receivers, and C_{all}, B, Full and Replay in each receiver (Section 4.4). SC_{exact} rewrites every source K/V tensor after SC. Elementwise checks find zero source discrepancies. Table 5 records the sequence identities discussed in Section 4.4. They concern skill-position KV, not all residual states. Replay can answer differently: Qwen3-8B/TheoremQA scores 62/63 versus the donor's 61/63.

Paired uncertainty. Table 6 uses 10,000 whole-skill-group bootstrap resamples with percentile intervals. The seed is 20261024 for the seven new cells. Qwen3-8B/MedCalc reuses its audited analysis. Intervals are exploratory and unadjusted for multiple comparisons, with few groups in some cells. Adversarial scoring of capped outputs leaves Qwen3-8B/MedCalc's early-minus-late gap at 63.7/64.5 points in Wrong/Neutral.

Additional Qwen3-8B/MedCalc content and layer controls. With all-layer source coverage, correct-skill KV gives 29.8/25.0% accuracy in Wrong/Neutral, unrelated-skill KV gives 4.8/6.5%, and a second predeclared neutral text's KV gives 1.6/4.0%. Eight-layer writes at 0–7, 8–15, 16–23 and 24–31 give 3.2/3.2/1.6/4.0% in Wrong and 0.8/4.0/4.8/5.6% in Neutral. These limited recoveries cannot identify minimum coverage by comparison with all 36 layers because the write budgets differ.

Audit. Independent processes verify hashes, conditions, cache writes, token identities and both scoring orders. Labels reproduce accuracies, cap counts and intervals for 10,944 new and 6076 reused records. Gates are reused.